

Qian ZhongJun 301541827

For our IAT 355 final project, I was responsible for end-to-end system design, real-world data engineering, live backend integration, and final dashboard implementation. My contributions reflect not only software development but also network configuration, physical infrastructure planning, and cybersecurity-aware interaction design.

Research Design & Infrastructure Planning

I originated the concept of simulating a live SIEM (Security Information and Event Management) system using VLAN-isolated VMs. I proposed and built a red-blue team lab environment using Kali Linux VMs on two separate laptops and a TL-SG105E switch with PVID configurations, as documented in IAT 355 Blue.docx, IAT 355 Red.docx, and our README.md

Key Deliverables:

- Full documentation of VLAN topology and RoE acquisition process
- Manual IP subnet configuration for safe, offline testing
- Realistic data capture including controlled ICMP floods and SYN scans
- Scripting and running tshark and hping3 simultaneously across multiple interfaces

Backend Development (Data Engineering & Live Integration)

I wrote the serve_json.py script and SSH port-forwarding setup to bridge the backend Python stream with Observable's frontend over port 8080x serve_json & README. This included:

- Converting .pcap traffic into a rolling JSON buffer using tshark
- Deploying a Flask app to serve live JSON packets
- Maintaining a 2-minute sliding buffer (deque) for continuous packet display
- Using SSH tunnels for secure cross-device data forwarding

Commands used:

```
sudo tshark -i eth0.10 -T fields -e frame.time_epoch -e ip.src -e ip.dst -e ip.proto ...
```

```
ssh -L 8081:localhost:8080 kali@192.168.1.x
```

```
python3 serve_json.py
```

Visualization Engineering in Observable

I wrote and structured the final dashboard notebook (56ba25a95f74523d@661.js) that features:

- Five coordinated views (line, pie, two bar charts, and a dot plot)
- VM filtering and protocol dropdown menus
- Cross-filtering through timeBrush and interactive signal listeners

Visual Layers I implemented:

- Time Series: Line chart of packet volume by protocol and VM
- Protocol Pie Chart: Shows load distribution across sources
- Destination Port Group Analysis: Horizontal bar chart segmented by protocol
- TCP Flag Pattern: Bar chart for detecting SYN/ACK flooding behavior
- Source IP Dot Plot: Detailed interaction per sender

All charts are synchronized and update based on brushing and VM/protocol filters.

This dashboard is fed live by the Python backend, enabling SOC-style streaming analysis

Cybersecurity Lab Execution & Data Curation

I personally:

- Set up and tested Armitage, Nmap, and Hping3 on the red team VM
- Simulated:
 - ICMP flood attacks (hping3 --icmp)
 - Nmap slow scans (nmap -sS -T2)
 - SYN floods (hping3 -S --flood)
- Captured and converted .pcap to .csv using tshark
- Carefully curated datasets to stay under 10,000 packets, ensuring smooth Observable processingPoC data cap

These were compiled into attack_syn.csv, attack_icmp.csv, and others used for the static and live demos.

Entire lab has maintained professional safety standards: isolated VLAN, no Wi-Fi, RoE confirmation, midnight test hours, and packet volume moderation.